

1 to 3

```
!pip install numpy pandas scikit-learn matplotlib seaborn openpyxl
```

4

```
import pandas as pd
```

```
# Automatically generating the files so you don't have to [cite: 79, 85]
```

```
pd.DataFrame({'Name': ['Srikanth', 'Snigdha', 'Mary'], 'Age': [28, 22, 31], 'Score': [85, 78, 92]}).to_csv('sample_data.csv', index=False)
```

```
pd.DataFrame({'Name': ['Rajesh', 'Ramesh', 'Swati', 'Florina', 'Pooja', 'Raghu'], 'Course': ['BCA', 'BCA', 'BCOM', 'BCOM', 'BBA', 'BBA'], 'Sem': [1, 2, 1, 3, 2, 4]}).to_excel('sample_data.xlsx', index=False)
```

```
# Exact syllabus code starts here[cite: 88, 90, 91, 93, 97]:
```

```
csv_file_path = 'sample_data.csv'
```

```
excel_file_path = 'sample_data.xlsx'
```

```
data_csv = pd.read_csv(csv_file_path)
```

```
print("CSV File Data:")
```

```
print(data_csv)
```

```
data_excel = pd.read_excel(excel_file_path)
```

```
print("\nExcel File Data:")
```

```
print(data_excel)
```

```
print("\nData Descriptions:")
```

```
print("CSV Data Description:")
```

```
print(data_csv.describe())
```

```
print("\nExcel Data Description:")
```

```
print(data_excel.describe())
```

```
print("\nData Types in CSV File:")
```

```
print(data_csv.dtypes)
```

```
print("\nData Types in Excel File:")
```

```
print(data_excel.dtypes)
```

```
5
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
# Automatically generating the file [cite: 204]
```

```
pd.DataFrame({'Student ID': range(1, 14), 'Study Hours': [5, 2, 8, 1, 3, 4, 9, 6, 7, 0.5, 10, 0, 12], 'Exam  
Score': [82, 48, 90, 35, 50, 66, 95, 75, 88, 30, 96, 20, 98]}).to_csv('study_data.csv', index=False)
```

```
# Exact syllabus code starts here[cite: 219, 222, 224]:
```

```
data = pd.read_csv('study_data.csv')
```

```
plt.figure(figsize=(14, 7))
```

```
# Scatter Plot [cite: 225, 226]
```

```
plt.subplot(1, 2, 1)
```

```
plt.scatter(data['Study Hours'], data['Exam Score'], color='dodgerblue', edgecolor='k', alpha=0.7)
```

```
plt.title('Study Hours vs. Exam Scores')
```

```
plt.xlabel('Study Hours')
```

```
plt.ylabel('Exam Scores')
```

```
plt.grid(True)
```

```
# Bar chart setup [cite: 232, 233, 234, 236]
```

```
bins = [0, 2, 4, 6, 8, 10, 12]
```

```
labels = ['0-2', '2-4', '4-6', '6-8', '8-10', '10-12']
```

```
data['Study Hour Range'] = pd.cut(data['Study Hours'], bins=bins, labels=labels, right=False)
```

```
grouped_data = data.groupby('Study Hour Range')['Exam Score'].mean()
```

```
# Bar chart plot [cite: 241, 244, 245]
```

```
plt.subplot(1, 2, 2)
```

```
grouped_data.plot(kind='bar', color='salmon')
```

```
plt.title('Average Exam Score by Study Hour Range')
```

```
plt.xlabel('Study Hour Range')
```

```
plt.ylabel('Average Exam Score')
```

```
plt.xticks(rotation=0)
```

```
plt.tight_layout()
```

```
plt.show()
```

6

```
import pandas as pd
```

```
from sklearn.impute import SimpleImputer
```

```
from sklearn.preprocessing import OneHotEncoder, StandardScaler
```

```
# Create dummy data [cite: 260, 268]
```

```
data = {
```

```
    'Age': [25, 30, None, 28, 35],
```

```
    'Gender': ['Female', 'Male', 'Male', 'Female', 'Male'],
```

```
'Income': [50000, 60000, 45000, None, 70000]

}

df = pd.DataFrame(data)


# Handling missing data [cite: 269, 270, 271]

imputer = SimpleImputer(strategy='mean')

df[['Age', 'Income']] = imputer.fit_transform(df[['Age', 'Income']])


print("Data after handling missing values:")

print(df)


# Encoding categorical variables [cite: 275, 276, 277]

encoder = OneHotEncoder()

encoded_data = encoder.fit_transform(df[['Gender']]).toarray()

encoded_df = pd.DataFrame(encoded_data, columns=encoder.get_feature_names_out(['Gender']))


print("\nData after categorical encoding:")

print(encoded_df)


# Feature scaling [cite: 284, 285, 287]

scaler = StandardScaler()

scaled_data = scaler.fit_transform(df[['Age', 'Income']])

scaled_df = pd.DataFrame(scaled_data, columns=['Scaled Age', 'Scaled Income'])


print("\nData after feature scaling:")

print(scaled_df)
```

```

import numpy as np

from sklearn.model_selection import train_test_split

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import accuracy_score


# Dummy student data [cite: 326, 328]

X = np.array([[88, 75], [95, 98], [60, 50], [45, 30], [38, 40], [85, 95], [78, 60], [50, 55], [40, 45], [60,
70]])

y = np.array([1, 1, 0, 0, 0, 1, 1, 0, 0, 1])


X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)


# Initialize and train [cite: 331, 332, 334]

knn = KNeighborsClassifier(n_neighbors=3)

knn.fit(X_train, y_train)


y_pred = knn.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print("Accuracy on the test set: {:.2f}".format(accuracy))


# Take user input [cite: 340, 342, 344]

exam_score1 = float(input("Enter Exam Score 1: "))

exam_score2 = float(input("Enter Exam Score 2: "))

user_input = np.array([[exam_score1, exam_score2]])


predicted_outcome = knn.predict(user_input)

```

```

if predicted_outcome[0] == 1:

    print("Based on the exam scores provided, the student is predicted to pass.")

else:

    print("Based on the exam scores provided, the student is predicted to fail.")

```

8

```

import numpy as np

from sklearn.linear_model import LinearRegression

# Dummy house price prediction data [cite: 382, 384]

X = np.array([[1000, 2], [1500, 3], [1200, 2], [1800, 4], [900, 2], [2000, 3]])

y = np.array([300000, 400000, 350000, 500000, 280000, 450000])

model = LinearRegression()

model.fit(X, y)

# User input [cite: 388, 389, 390]

size = float(input("Enter the size of the house in sqft: "))

bedrooms = int(input("Enter the number of bedrooms: "))

new_data = np.array([[size, bedrooms]])

predicted_price = model.predict(new_data)

print("Predicted price for a house with size {} sqft and {} bedrooms: Rs.{:.2f}".format(size, bedrooms,
predicted_price[0]))

```

9

```

import numpy as np

```

```

from sklearn.tree import DecisionTreeClassifier, plot_tree

from sklearn.tree import export_text

import matplotlib.pyplot as plt


# Custom dummy data for fruit classification [cite: 420, 421]

X = np.array([[150, 0], [170, 1], [120, 0], [140, 1], [200, 1], [130, 0]])

y = np.array(['Apple', 'Orange', 'Apple', 'Orange', 'Melon', 'Apple'])


clf = DecisionTreeClassifier(random_state=42)

clf.fit(X, y)


# Visualize rules [cite: 426, 427]

tree_rules = export_text(clf, feature_names=['Weight', 'Texture'])

print("Decision Tree Classifier Rules:\n", tree_rules)


# Plot Tree [cite: 429, 430]

plt.figure(figsize=(10, 6))

plot_tree(clf, filled=True, feature_names=['Weight', 'Texture'], class_names=np.unique(y))

plt.show()

```

```

10

import numpy as np

import matplotlib.pyplot as plt

from sklearn.cluster import KMeans


# Generate dummy customer data

X = np.array([[30, 50000], [35, 60000], [40, 80000], [25, 30000], [45, 100000],

```

```
[20, 20000], [50, 120000], [55, 150000], [60, 140000], [28, 40000]]])
```

```
kmeans = KMeans(n_clusters=3, random_state=0)
```

```
kmeans.fit(X)
```

```
labels = kmeans.labels_
```

```
centers = kmeans.cluster_centers_
```

```
# Visualize [cite: 462, 463, 464]
```

```
plt.figure(figsize=(8, 6))
```

```
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis', s=50, alpha=0.8)
```

```
plt.scatter(centers[:, 0], centers[:, 1], c='red', s=200, marker='X', label='Centroids')
```

```
plt.xlabel('Age')
```

```
plt.ylabel('Income')
```

```
plt.title('K-Means Clustering of Customers')
```

```
plt.legend()
```

```
plt.show()
```